

K-디지털 트레이닝

AOP

[KB] IT's Your Life

✓ AOP Aspect Oriented Programming

○ 관점(Aspect)지향 프로그래밍

AOP는 프로그램에서 핵심 기능과 공통 기능을 분리하여 관리할 수 있게 하는 프로그래밍
특정 페이지에서 구현되어야 하는 핵심 기능과 여러 페이지에서 공통으로 나타나는 기능을 분리
공통적으로 사용되는 기능은 하나의 클래스 파일로 모아두었다가, 실행 시 필요한 곳에 끼워 넣는
방식을 사용

○ 관심사의 예

- 이 작업을 하는 사용자가 적절한 권한을 가진 사용자인가?
- 이 작업에서 발생할 수 있는 모든 예외는 어떻게 처리해야 하는가?

○ 관심사의 분리

- 개발자가 염두에 두어야 하는 일들은 별도의 관심사로 분리하고,
- **핵심 비즈니스 로직만을 작성할 것을 권장**

→ 기존의 코드(핵심 비즈니스 로직)를 수정하지 않고, 원하는 기능(관심사)들과 결합

✓ AOP Aspect Oriented Programming

A사이트

로그인

개인정보보기

로그아웃

B사이트

물건검색

장바구니

로그인

주문

로그아웃

C사이트

로그인

주문정보보기

확인

로그아웃

관점클래스

로그인()
로그아웃()

C사이트 : Order interface
COrder.java - 주문정보보기 (orderView())
확인 (confirm())

B사이트 : Product interface
BProduct.java - String 물건검색 (search())
장바구니 (basket())
주문 (orderProduct())

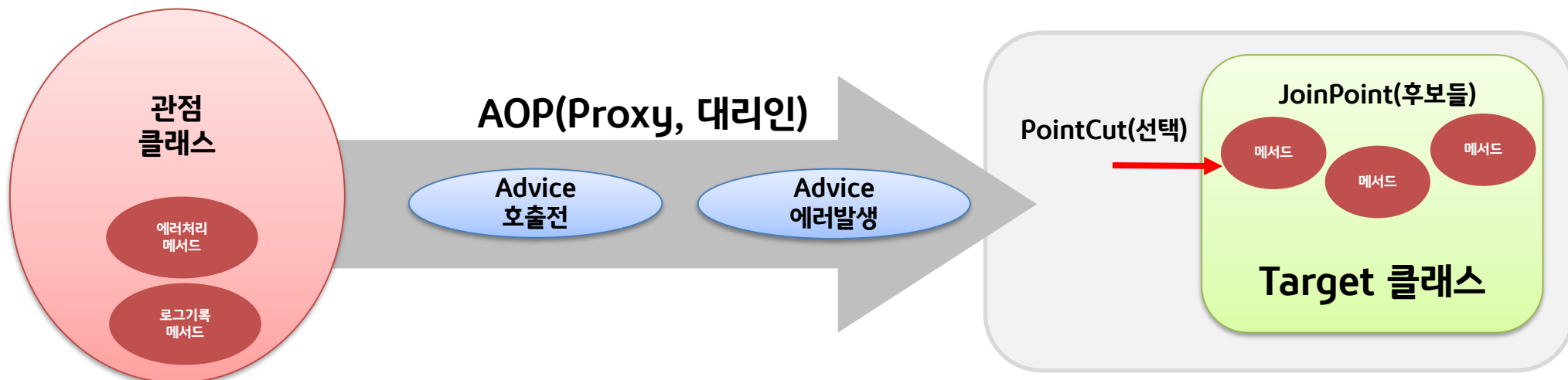
1. 로그인, 로그아웃을 조립!
2. 장바구니에 error => aop로 에러처리!
3. 물건검색 retrun 이 있는 aop메시지 호출!

✓ AOP Aspect Oriented Programming

- **공통 관점과 핵심 관점을 나누어 프로그래밍**
 - 프로그램을 핵심 기능(핵심 관점)과 공통 기능(공통 관점)으로 나누어 작성
- **공통 관점에 해당하는 기능을 묶어 별도의 클래스로 만든다**
 - 공통 기능에 해당하는 부분들을 별도의 클래스에 모아둔다.
- **핵심 관점을 가진 클래스를 만든다**
 - 핵심 기능을 담당하는 클래스를 별도로 작성
- **실행 시 핵심 관점 사이에 공통 관점을 끼워 넣는다**
 - 실행 시 필요한 곳에 공통 기능을 삽입

✓ AOP 용어들

- **관점(Aspect):** 교차 관심사를 모듈화한 코드 블록으로, 여러 핵심 비즈니스 로직에 통합될 수 있음.
 - 예를 들어 보안 검증 로직은 여러 서비스에 걸쳐 존재하는 공통 로직이므로, 이를 하나의 관점으로 관리
- **조언(Advice):** 어느 특정 시점에 공통 관점 메서드를 호출할 것인지 조언
- **조인포인트(Join Point):** 조언이 적용될 수 있는 프로그램 실행의 특정 지점.(후보들)
 - 메서드 호출, 객체 생성 등 다양한 프로그램 실행 후보 시점
- **포인트컷(Pointcut):** AOP에서 조인포인트 중 특정하게 지정한 지점
 - 포인트컷에 따라 어느 부분에 조언이 적용될지 결정
- **위빙(Weaving):** 실제 코드에 관점을 삽입하는 과정
 - 컴파일 타임, 로드 타임, 또는 런타임 중 발생할 수 있음. Spring AOP는 런타임만 지원

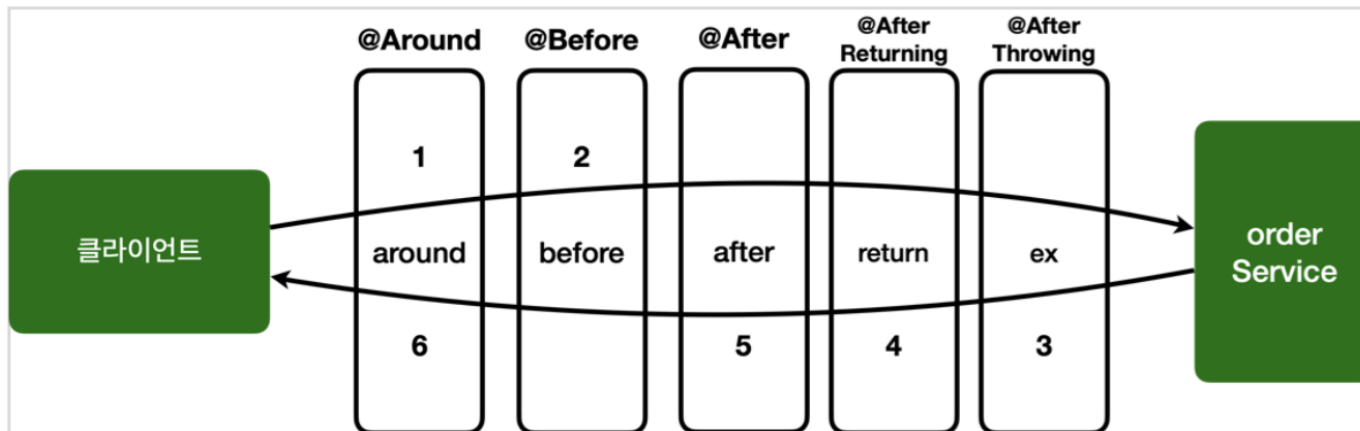


✓ AOP 용어들

○ Advice(충고, 언제 조립할 것인지 시점 지정)

- 분리된 관심사 코드(메서드)
- Advice가 동작되는 위치를 어노테이션으로 지정

어노테이션	설명
@Around	메서드 실행 전/후로 실행
@Before	JointPoint 호출 전 실행
@AfterReturning	모든 실행이 정상적으로 이루어진 후에 실행
@AfterThrowing	예외가 발생한 뒤에 실행
@After	정상적으로 실행되거나 예외가 발생했을 때 구분없이 실행되는 코드



✓ AOP 용어들

- Pointcut : Advice를 어떤 JoinPoint에 결합할 것인지 결정하는 표현식

구분	설명
execution(실행대상지정)	메서드를 기준으로 Pointcut을 설정
within(@within)	특정 타입(클래스)을 기준으로 Pointcut을 설정
this	주어진 인터페이스를 구현한 객체를 대상으로 Pointcut을 지정
args(@args)	특정 파라미터를 가지는 대상들만 Pointcut으로 설정
@annotation	특정한 어노테이션이 적용된 대상들만을 Pointcut으로 설정

- 예: Advice + Pointcut

```

- @Before("execution(* org.scoula.sample.service.SampleService*.*(..))")
  public void logBefore() {
    ...
  }

```

리턴타입 제한없음

지정한 이름으로 시작하는 클래스명

매개변수 제한없음

메서드명 제한없음

✓ 의존 라이브러리

implementation 'org.aspectj:aspectjrt:1.9.20'

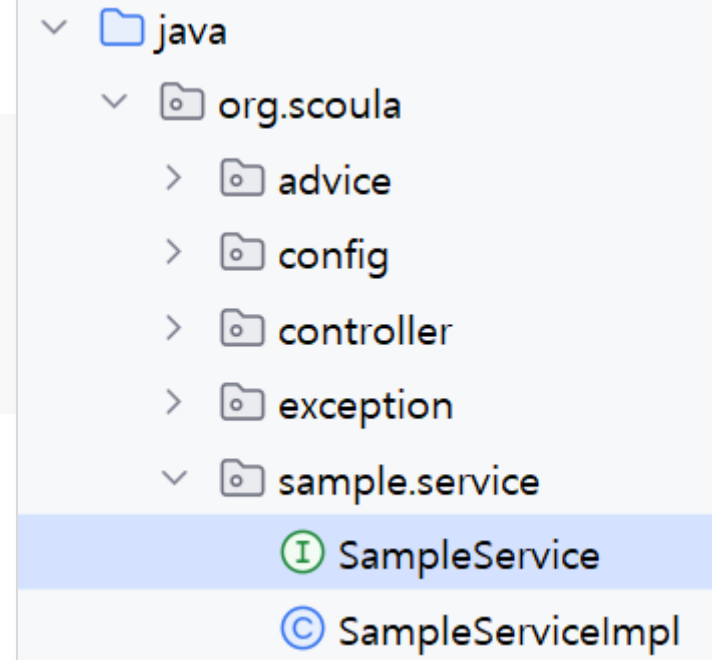
implementation 'org.aspectj:aspectjweaver:1.9.20'

✓ 프로젝트

- 템플릿: SpringLegacy
- name: aopex

SampleService.java

```
package org.scoula.sample.service;  
  
public interface SampleService {  
    public Integer doAdd(String str1, String str2) throws Exception;  
}
```



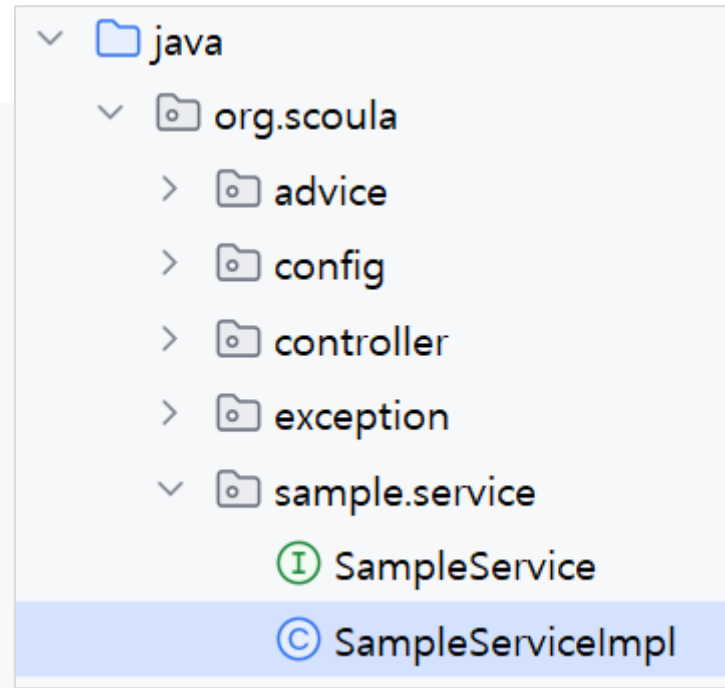
SampleServiceImpl.java

```
package org.scoula.sample.service;

import org.springframework.stereotype.Service;

@Service
public class SampleServiceImpl implements SampleService {

    @Override
    public Integer doAdd(String str1, String str2) throws Exception {
        return Integer.parseInt(str1) + Integer.parseInt(str2);
    }
}
```



LogAdvice

```
package org.scoula.advice;

import lombok.extern.log4j.Log4j;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;
```

@Aspect

@Log4j

@Component

```
public class LogAdvice {
```

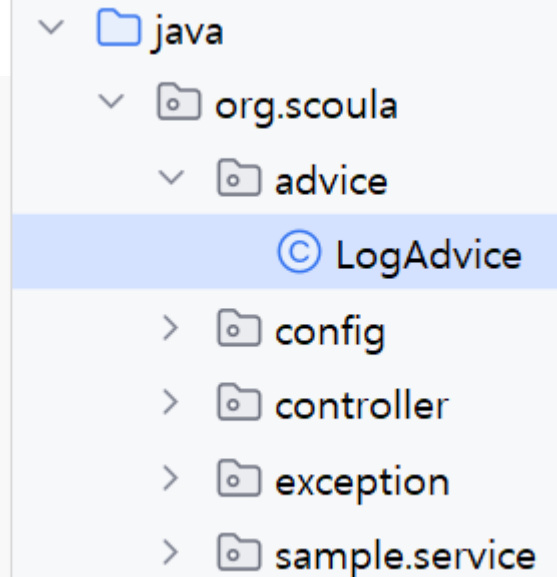
```
    @Before("execution(* org.scoula.sample.service.SampleService*.*(..))")
```

```
    public void logBefore() {
```

```
        System.out.println("=====");
```

```
    }
```

```
}
```



Rootconfig

```
@Configuration
@ComponentScan(basePackages = {
    "org.scoula.advice",
    "org.scoula.sample.service"
})
@EnableAspectJAutoProxy //aop활성화
public class RootConfig {

}
```

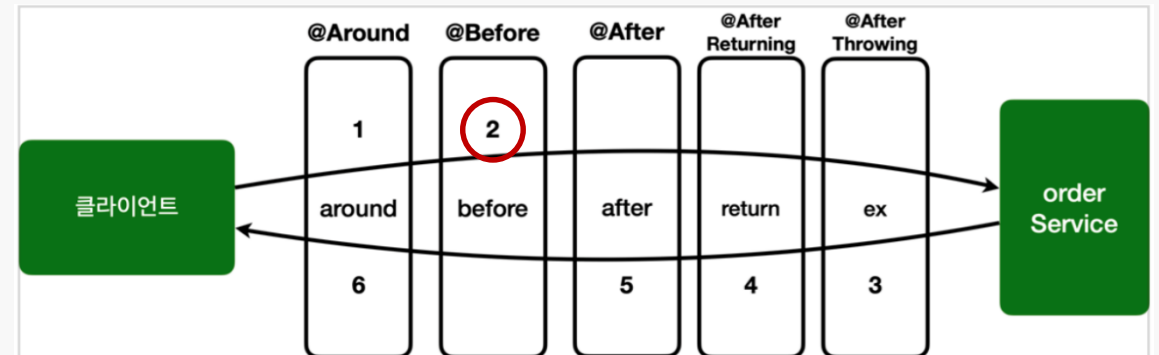
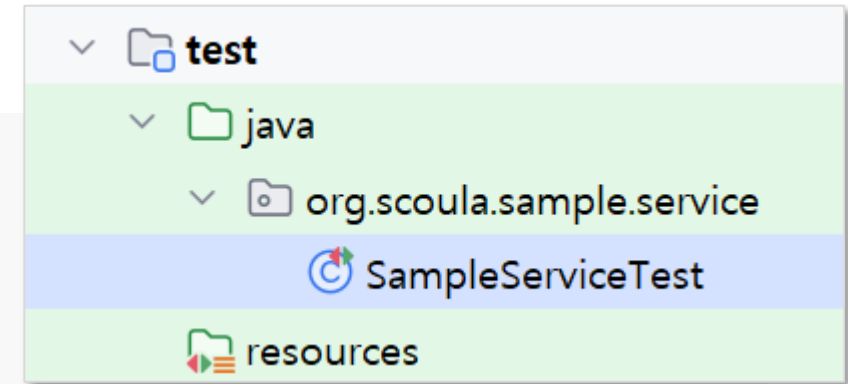
test::SampleServiceTest.java

```
package org.scoula.sample.service;

...
@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = { RootConfig.class })
@Log4j
class SampleServiceTest {

    @Autowired
    private SampleService service;

    @Test
    public void doAdd() throws Exception {
        System.out.println(service.doAdd("123", "456"));
    }
}
```



✓ args를 이용한 파라미터 추적

- 해당 메서드에 전달되는 파라미터가 무엇인지 기록하건, 예외가 발생했을 때 어떤 파라미터에 문제가 있는지 알고 싶은 경우
- Pointcut에 args를 이용한 파라미터 추적 설정 추가

```
...
public class LogAdvice {

    @Before("execution(* org.scoula.sample.service.SampleService*.doAdd(String, String)) && args(str1, str2)")
    public void logBeforeWithParam(String str1, String str2) {
        System.out.println("str1:" + str1);
        System.out.println("str2:" + str2);
    }

}
```

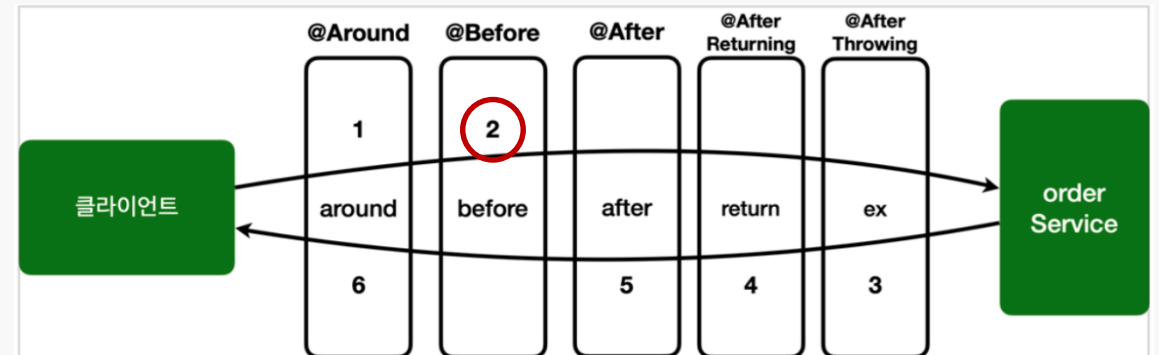
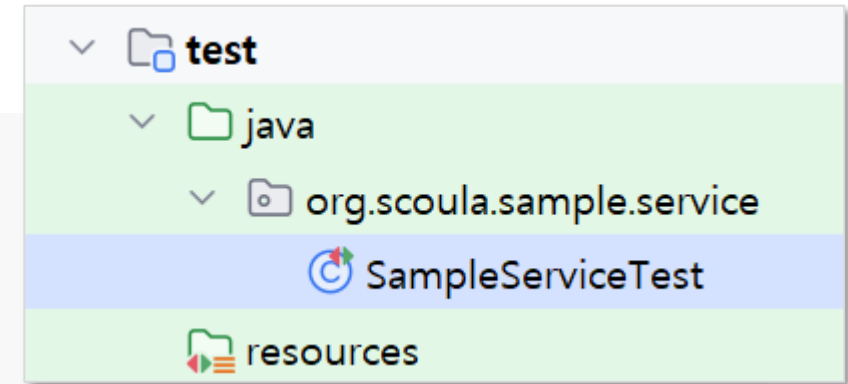
test::SampleServiceTest.java

```
package org.scoula.sample.service;

...
@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = { RootConfig.class })
@Log4j
class SampleServiceTest {

    @Autowired
    private SampleService service;

    @Test
    public void doAdd() throws Exception {
        System.out.println(service.doAdd("123", "456"));
    }
}
```



=====

str1:123

str2:456

579

✓ @AfterThrowing

- 지정된 대상이 예외를 발생한 후에 동작

```
@Aspect
@Log4j
@Component
public class LogAdvice {
    ...

    @AfterThrowing(pointcut = "execution(* org.scoula.sample.service.SampleService*.*(..))", throwing="exception")
    public void logException(Exception exception) {
        System.out.println("Exception...!!!!");
        System.out.println("exception: " + exception);
    }
}
```

test::SampleServiceTest.java

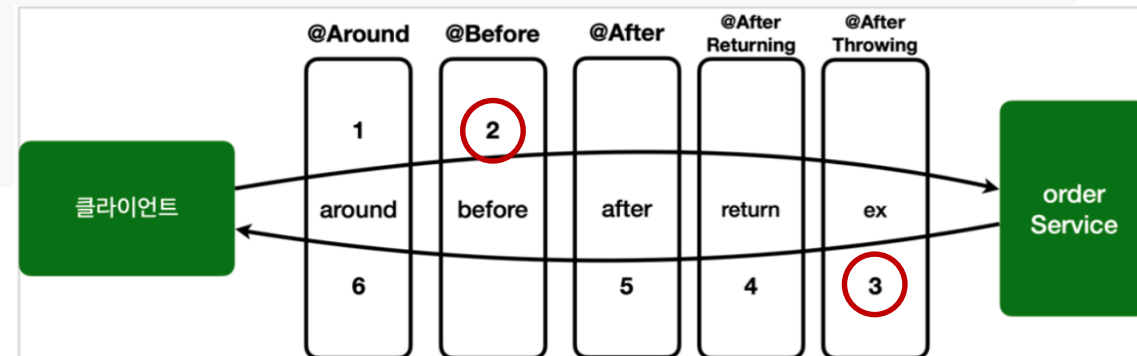
```
public class SampleServiceTest {  
    ...  
  
    @Test  
    public void addError() throws Exception {  
        System.out.println(service.doAdd("123", "ABC"));  
    }  
}
```

=====

str1:123
str2:ABC

Exception...!!!!

exception: java.lang.NumberFormatException: For input string: "ABC"



✓ @Around와 ProceedingJoinPoint

- 메서드의 실행 전과 실행 후에 처리가 가능

```
...
public class LogAdvice {

    @Around("execution(* org.scoula.sample.service.SampleService*.*(..))")
    public Object logTime(ProceedingJoinPoint pjp) {
        long start = System.currentTimeMillis();

        System.out.println("Target: " + pjp.getTarget());
        System.out.println("Param: " + Arrays.toString(pjp.getArgs()));

        Object result = null;
        try {
            result = pjp.proceed(); // 실제 메서드 호출
        } catch (Throwable e) {
            e.printStackTrace();
        }
    }
}
```

✓ @Around와 ProceedingJoinPoint

```
long end = System.currentTimeMillis();

    System.out.println("TIME: " + (end - start));

    return result;
}
```

Target: org.scoula.sample.service.SampleServiceImpl@74971ed9

Param: [123, 456]

=====

str1:123

str2:456

TIME: 3

579

